

Univerzitet u Beogradu
Fakultet organizacionih nauka
Katedra za informacione sisteme

Implementacija informacionih sistema

APLIKACIJA ZA REZERVACIJU TERMINA U SALONU LEPOTE TRAČ

A.1 aplikacija za rezervacije i A.2 portal za izveštavanje

Tamara Jerotijević 2022/0362
Jovana Dumić 2023/1017

1. PMOV model podataka

PMOV model predstavlja logičku osnovu sistema za rezervaciju termina u salonu lepote „Trač“. Model je podeljen na dva dela, jer se u varijanti arhitekture C aplikacija A.1 i portal A.2 oslanjaju na zasebne baze podataka. A.1 čuva operativne podatke potrebne za rad salona i upravljanje rezervacijama, dok A.2 čuva podatke prilagođene izveštavanju i popunjava ih asinhrono na osnovu RabbitMQ događaja.

Centralno mesto u A.1 modelu zauzima entitet Rezervacija. On čuva identifikacione podatke korisnika (ime, prezime, adresu, poštanski broj, mesto, državu i email), pristupnu šifru, status rezervacije, izabranu valutu, kurs, procenat popusta, ukupnu i konačnu cenu, kao i datum kreiranja i eventualnog otkazivanja.

Jedna rezervacija sadrži jednu ili više stavki predstavljenih entitetom StavkaRezervacije. Svaka stavka povezuje rezervaciju sa jednom uslugom i čuva izabrani datum, vreme početka termina i cenu usluge u trenutku rezervacije. Na taj način promena cene usluge u administraciji ne menja cenu već kreirane stavke.

Entitet Usluga opisuje jednu uslugu salona i sadrži naziv, opis, trajanje u minutima, maksimalan broj klijenata po terminu, vreme početka prvog termina, vreme završetka poslednjeg termina i cenu. Svaka usluga pripada tačno jednoj kategoriji, dok jedna KategorijaUsluge može da sadrži više usluga.

Entitet OsnovneInformacije sadrži naziv salona, lokaciju, opis, radno vreme i datum do kada važi popust od 10%. Ovi podaci se prikazuju na početnoj strani, a osnovne informacije se keširaju u Redis-u kako bi se smanjio broj ponovljenih čitanja iz baze.

DozvoljenaValuta predstavlja administrativno definisanu listu valuta koje korisnik može da izabere prilikom rezervacije. Rezervacija čuva oznaku izabrane valute i kurs koji je korišćen u trenutku konačnog obračuna. Kurs se dobija pozivanjem eksternog Frankfurter API servisa.

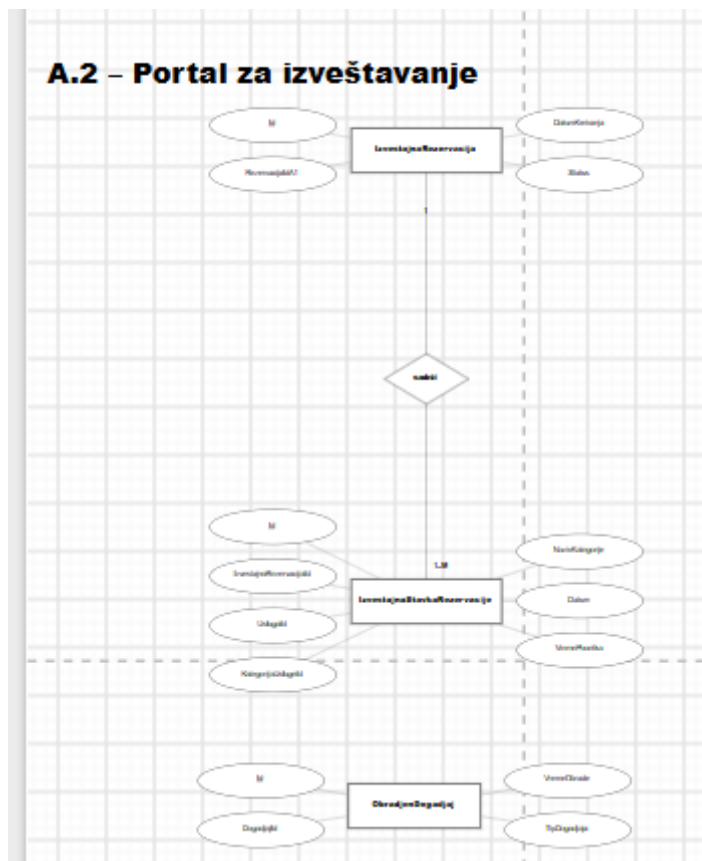
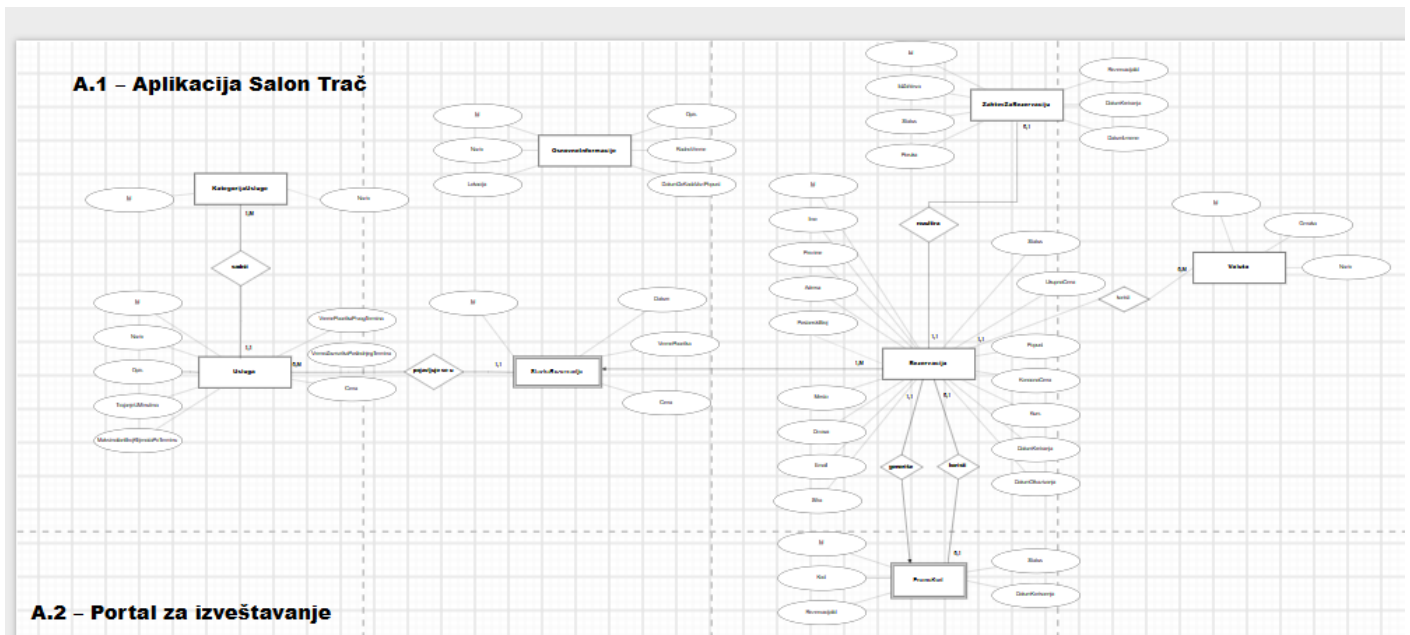
Sistem lojalnosti je predstavljen entitetom PromoKod. Svaka uspešno kreirana rezervacija generiše jedan jedinstveni promo-kod. Kod može biti DOSTUPAN, ISKORISCEN ili NEVAZEĆI. Dostupan kod omogućava dodatnih 5% popusta na sledeću rezervaciju. Kada se kod iskoristi, beleži se promena statusa i datum korišćenja. Ako se rezervacija koja je generisala kod otkaže, neiskorišćen kod postaje nevažeći.

Entitet ZahtevZaRezervaciju služi za praćenje asinhronne obrade nove rezervacije. API pri prijemu zahteva ne upisuje odmah samu rezervaciju, već kreira zapis sa jedinstvenim IdZahteva i statusom obrade. Kada background proces završi obradu, zapis se povezuje sa kreiranom rezervacijom ili dobija status odbijanja i odgovarajuću poruku.

A.2 portal za izveštavanje ima zaseban, pojednostavljen model. Entitet IzvestajnaRezervacija čuva identifikator rezervacije iz A.1 sistema, originalni datum kreiranja i status. Sa njim je u vezi jedan-prema-više entitet IzvestajnaStavkaRezervacije, koji čuva identifikatore usluge i kategorije, naziv kategorije, datum i vreme početka rezervisanog termina.

Entitet ObradjenDogadjaj služi za idempotentnu obradu RabbitMQ događaja. Čuva identifikator događaja, tip događaja i vreme obrade, čime se sprečava da ponovljena isporuka iste poruke dovede do dupliranja podataka u A.2 bazi.

Između A.1 i A.2 baze ne postoje direktne relacije niti strani ključevi. Sinhronizacija se vrši isključivo događajima rezervacija objavljenim na RabbitMQ-u. Za izveštaj po kategorijama koriste se stavke rezervacija koje nisu otkazane, dok se za izveštaj po datumima svaka rezervacija broji tačno jednom prema originalnom datumu kreiranja.



2. Opis slučajeve korišćenja

SK1: Pregled osnovnih informacija i usluga

Akter: Posetilac

Preduslov: Aplikacija A.1 je pokrenuta, a osnovni podaci salona su uneti.

Postuslov: Posetiocu su prikazane osnovne informacije salona i usluge grupisane po kategorijama.

Osnovni scenario:

1. Posetilac otvara početnu stranicu aplikacije.
2. Sistem pokušava da učitava osnovne informacije salona iz Redis keša.
3. Ako podaci nisu u kešu, sistem ih učitava iz baze podataka i smešta u Redis.
4. Sistem učitava kategorije usluga i sve definisane usluge.
5. Početna strana prikazuje naziv salona, lokaciju, opis i radno vreme.
6. Usluge se prikazuju grupisane po pripadajućim kategorijama, zajedno sa opisom, cenom, trajanjem i rasponom termina.

Alternativni scenariji:

- 3.1. Ako osnovne informacije još nisu unete, sistem prikazuje obaveštenje da podaci nisu dostupni.
- 4.1. Ako nema definisanih kategorija ili usluga, sistem prikazuje odgovarajuću poruku umesto prazne liste.

SK2: Administracija podacima salona

Akter: Administrator

Preduslov: Aplikacija je pokrenuta i povezana sa A.1 bazom podataka i Redis kešom.

Postuslov: Izabrani administrativni podaci su sačuvani, a prikaz aplikacije koristi nove vrednosti.

Osnovni scenario:

1. Administrator otvara odgovarajuću stranicu administracije.
2. Sistem učitava postojeće osnovne informacije, kategorije usluga, usluge, dozvoljene valute i datum do kada važi popust od 10%.
3. Administrator dodaje ili menja osnovne informacije, kategoriju, uslugu, valutu ili datum popusta.
4. Sistem validira unete podatke.
5. Sistem čuva promene u relacionoj bazi podataka.
6. Ako su izmenjene osnovne informacije salona, sistem uklanja stari zapis iz Redis keša kako bi naredno čitanje učitalo sveže podatke.
7. Sistem prikazuje poruku o uspešnoj izmeni.

Alternativni scenariji:

- 3.1. Administrator briše kategoriju, uslugu ili dozvoljenu valutu.
- 3.2. Kategorija koja sadrži usluge ne može da se obriše; sistem odbija zahtev i prikazuje poruku.
- 4.1. Ako je vreme početka prvog termina veće ili jednako vremenu završetka poslednjeg termina, sistem odbija unos usluge.
- 4.2. Ako valuta sa istom oznakom već postoji, sistem odbija duplikat.

SK3: Kreiranje rezervacije

Akter: Posetilac

Preduslov: Postoje definisane usluge i dozvoljene valute, a za najmanje jedan odabrani termin postoji slobodan kapacitet.

Postuslov: Ako su svi uslovi zadovoljeni, rezervacija je asinhrono sačuvana u bazi, generisane su pristupna šifra i novi promo-kod, a događaj o kreiranju je objavljen za A.2 portal.

Osnovni scenario:

1. Posetilac otvara stranicu za novu rezervaciju.
2. Sistem učitava dostupne usluge i dozvoljene valute.
3. Posetilac unosi ime, prezime, adresu, poštanski broj, mesto, državu i email.
4. Posetilac bira jednu ili više usluga.
5. Za svaku uslugu posetilac bira datum, nakon čega sistem učitava samo termine koji imaju slobodna mesta.
6. Posetilac bira termin za svaku stavku rezervacije.
7. Posetilac opciono unosi promo-kod za dodatnih 5% popusta.
8. Posetilac bira valutu u kojoj želi da vidi i sačuva konačan obračun.
9. Sistem poziva eksterni servis za kurs i prikazuje obračun cene, uz eventualni vremenski popust od 10% i promo popust od 5%.
10. Posetilac potvrđuje rezervaciju.
11. A.1 API formira komandu za kreiranje rezervacije, kreira tehnički zapis ZahtevZaRezervaciju sa statusom NA_CEKANJU i objavljuje komandu u RabbitMQ red „kreiranje-rezervacije“.
12. API odmah vraća identifikator zahteva, a frontend prelazi na stranicu za praćenje statusa obrade.
13. Salon.Radnik preuzima poruku iz reda, označava zahtev kao U_OBRADI i vrši finalne provere: postojanje usluga, dozvoljenu valutu, validnost termina i raspoloživ kapacitet.
14. Background proces ponovo računa cenu na osnovu cena iz baze, proverava promo-kod i pribavlja kurs za izabranu valutu.
15. Ako su svi uslovi zadovoljeni, proces generiše jedinstvenu pristupnu šifru i novi promo-kod, a rezervaciju i njene stavke upisuje u bazu u okviru transakcije.
16. Ako je korišćen važeći promo-kod, njegov status se menja u ISKORISCEN i beleži se datum korišćenja.
17. Nakon uspešnog upisa, objavljuje se događaj rezervacija.kreirana za portal A.2.

18. Zahtev za rezervaciju dobija status USPESNA i vezu ka kreiranoj rezervaciji.

19. Frontend periodično proverava status zahteva i, kada obrada uspe, prikazuje korisniku pristupnu šifru i generisani promo-kod.

Alternativni scenariji:

- 7.1. Ako promo-kod ne postoji ili nije dostupan, rezervacija se obračunava bez dodatnog promo popusta.
- 13.1. Ako je termin u međuvremenu popunjen, background proces odbija rezervaciju i zahtev dobija status ODBIJENA.
- 13.2. Ako izabrana usluga, valuta ili termin nisu validni, zahtev se odbija uz odgovarajuću poruku.
- 14.1. Ako eksterni servis za kurs nije dostupan, obrada rezervacije ne može da se završi i korisniku se vraća informacija o neuspešnoj obradi.
- 16.1. Ako je promo-kod u međuvremenu iskoristila druga rezervacija, zahtev se odbija jer kod više nije dostupan.

SK4: Izmena rezervacije

Akter: Korisnik

Preduslov: Rezervacija postoji, aktivna je i korisnik poseduje odgovarajuću kombinaciju email adrese i pristupne šifre.

Postuslov: Rezervacija sadrži izmenjen skup usluga, cena je preračunata, a događaj o izmeni je objavljen za A.2 portal.

Osnovni scenario:

1. Korisnik unosi email i pristupnu šifru dobijenu prilikom kreiranja rezervacije.
2. Sistem pronalazi rezervaciju i prikazuje njene osnovne podatke, status, cenu, promo-kod i postojeće stavke.
3. Korisnik bira dodavanje nove usluge ili uklanjanje postojeće usluge.
4. Ako se dodaje usluga, korisnik bira uslugu, datum i jedan od trenutno dostupnih termina.
5. Sistem proverava da rezervacija nije otkazana, da usluga postoji, da je termin validan i da ima slobodnih mesta.
6. Sistem dodaje novu stavku ili uklanja izabranu postojeću stavku.
7. Ukupna i konačna cena rezervacije se ponovo računaju na osnovu trenutnih stavki, sa sačuvanim procentom popusta rezervacije i kursom dobijenim za izabranu valutu.
8. Promena se čuva u bazi podataka u okviru transakcije.
9. A.1 objavljuje događaj rezervacija.izmenjena sa trenutnim stanjem rezervacije i stavki.
10. Frontend ponovo učitava rezervaciju i prikazuje ažurirane podatke.

Alternativni scenariji:

- 1.1. Ako kombinacija email adrese i šifre nije ispravna, sistem prikazuje poruku da rezervacija nije pronađena.
- 4.1. Ako je izabrani termin popunjen, sistem odbija dodavanje usluge.
- 6.1. Poslednja preostala usluga ne može da se ukloni; korisnik u tom slučaju mora da otkaže celu rezervaciju.
- 5.1. Otkazana rezervacija ne može da se menja.

SK5: Otkazivanje rezervacije

Akter: Korisnik

Preduslov: Rezervacija postoji, aktivna je i korisnik ima ispravan email i pristupnu šifru.

Postuslov: Rezervacija ima status OTKAZANA, ostaje sačuvana u istoriji i ne može se ponovo aktivirati; njen neiskorišćeni promo-kod postaje nevažeći.

Osnovni scenario:

1. Korisnik pristupa rezervaciji pomoću email adrese i pristupne šifre.
2. Sistem prikazuje podatke rezervacije.
3. Korisnik bira opciju za otkazivanje cele rezervacije i potvrđuje akciju.
4. Sistem proverava da je rezervacija trenutno aktivna.
5. Status rezervacije se menja u OTKAZANA i beleži se datum otkazivanja.
6. Ako je promo-kod koji je generisala rezervacija i dalje DOSTUPAN, njegov status se menja u NEVAZEĆI.
7. Rezervacija i njene stavke ostaju u bazi radi istorije i izveštavanja.
8. A.1 objavljuje događaj rezervacija.otkazana.
9. A.2 prima događaj i menja status odgovarajuće izveštajne rezervacije.

Alternativni scenariji:

- 1.1. Ako email i šifra nisu ispravni, sistem prikazuje poruku da rezervacija nije pronađena.
- 4.1. Ako je rezervacija već otkazana, sistem odbija ponovno otkazivanje.
- 6.1. Ako je promo-kod već ISKORISCEN, njegov status se ne menja jer je korišćenje već evidentirano.

SK6: Pregled broja rezervisanih termina po kategoriji usluge

Akter: Administrator / odgovorno lice salona

Preduslov: A.2 portal i njegov backend su pokrenuti, a A.2 baza sadrži podatke sinhronizovane RabbitMQ događajima.

Postuslov: Prikazan je ažuran broj trenutno rezervisanih termina grupisan po kategorijama usluga.

Osnovni scenario:

1. Administrator otvara A.2 portal za izveštavanje.
2. Frontend poziva A.2 API za izveštaj termina po kategorijama.
3. Backend grupiše izveštajne stavke prema kategoriji usluge.
4. Pri brojanju se zanemaruju stavke rezervacija čiji je status OTKAZANA.
5. Rezultat se sortira prema nazivu kategorije i vraća frontendu.
6. Frontend prikazuje tabelu sa nazivom kategorije i brojem rezervisanih termina.
7. Portal automatski ponavlja učitavanje podataka na svakih pet sekundi, bez ručnog osvežavanja stranice.

Alternativni scenariji:

- 2.1. Ako nema podataka za prikaz, portal prikazuje poruku da trenutno nema podataka.
- 2.2. Ako A.2 API nije dostupan, portal prikazuje poruku o grešci pri učitavanju izveštaja.

SK7: Pregled broja rezervacija po datumima

Akter: Administrator / odgovorno lice salona

Preduslov: A.2 portal je dostupan i A.2 baza sadrži sinhronizovane rezervacije.

Postuslov: Prikazan je hronološki broj rezervacija po danima, pri čemu se svaka rezervacija broji tačno jednom.

Osnovni scenario:

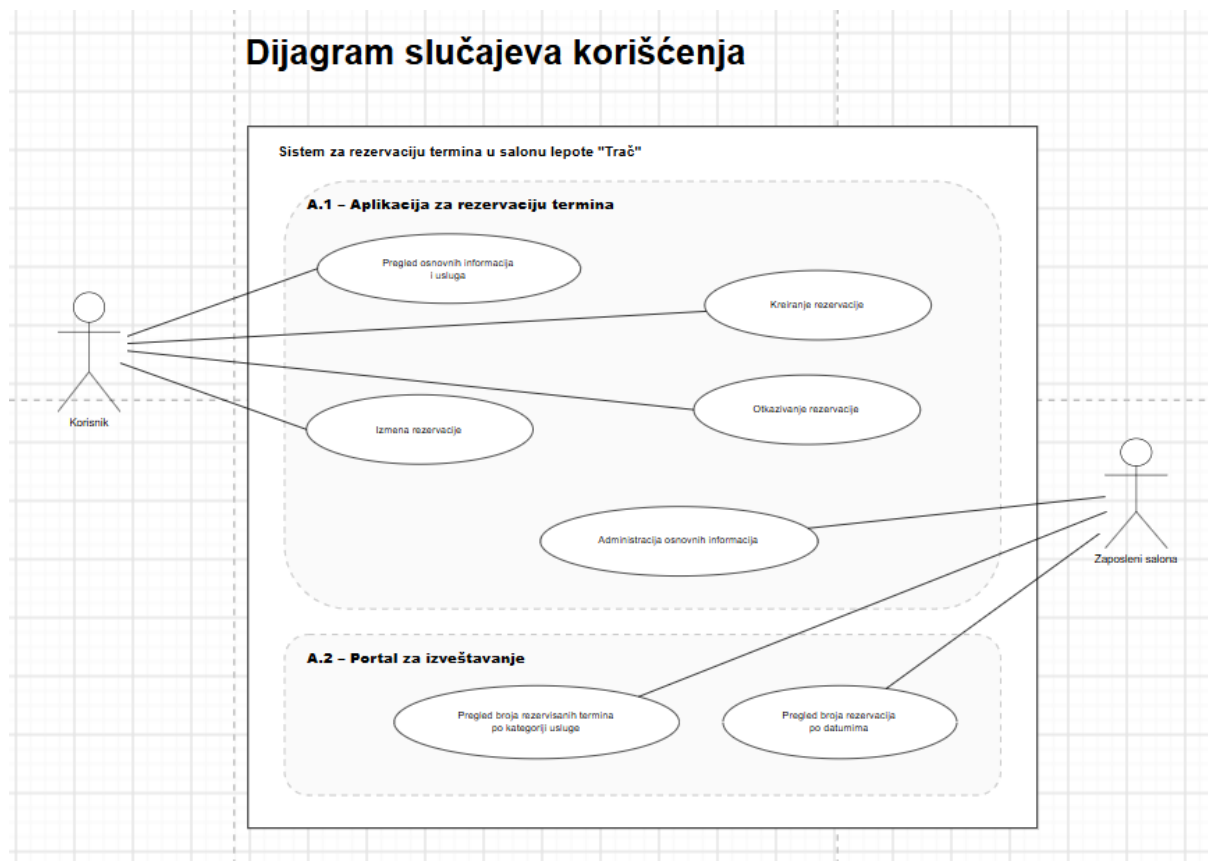
1. Administrator otvara A.2 portal.
2. Frontend poziva A.2 API za izveštaj rezervacija po datumima.
3. Backend grupiše IzvestajnaRezervacija zapise prema originalnom datumu kreiranja.
4. Svaka rezervacija se broji jednom, bez obzira koliko stavki odnosno usluga sadrži.
5. Izmene rezervacije ne menjaju originalni datum kreiranja, a otkazivanje ne briše rezervaciju iz A.2 baze.
6. Rezultat se sortira hronološki i vraća frontendu.
7. Frontend prikazuje tabelu sa datumom i brojem rezervacija.
8. Podaci se automatski osvežavaju na svakih pet sekundi.

Alternativni scenariji:

- 2.1. Ako nema rezervacija, portal prikazuje poruku da trenutno nema podataka.
- 2.2. Ako A.2 API nije dostupan, prikazuje se poruka o grešci.

3. Dijagram slučajeja korišćenja

Dijagram slučajeja korišćenja treba da prikaže tri osnovna tipa aktera: Posetioca, Korisnika koji već poseduje rezervaciju i Administratora odnosno odgovorno lice salona. Posetilac koristi pregled ponude i kreiranje rezervacije, Korisnik pristupa izmeni i otkazivanju postojeće rezervacije, dok Administrator upravlja podacima salona i koristi A.2 portal za oba izveštaja.



4. Dijagram sekvenci – SK3: Kreiranje rezervacije

Sekvenca kreiranja rezervacije prikazuje asinhroni tok karakterističan za varijantu arhitekture C. Učesnici koji treba da budu prikazani na dijagramu su: Posetilac, React frontend salon-web, Salon.Api, A.1 baza podataka, eksterni Frankfurter API, RabbitMQ red za kreiranje rezervacija, Salon.Radnik, RabbitMQ exchange za događaje, Izvestavanje.Api i A.2 baza podataka.

Tok počinje kada posetilac otvori stranicu za rezervaciju. Frontend učitava usluge i dozvoljene valute preko REST API-ja. Kada korisnik izabere uslugu i datum, frontend poziva API za dostupne termine. Salon.Api preko TerminService proverava broj već rezervisanih mesta i vraća samo termine za koje postoji slobodan kapacitet.

Kada korisnik izabere valutu, frontend poziva KursController. Backend najpre proverava da li je valuta dozvoljena, a zatim KursService poziva Frankfurter API i vraća trenutni kurs iz RSD u izabranu valutu. Za prikaz obračuna frontend može da zatraži i izračunavanje trenutne cene sa vremenskim popustom od 10% i opcionim promo-kodom od 5%.

Kada korisnik potvrdi rezervaciju, frontend šalje POST zahtev na /api/rezervacije. RezervacijeController formira KreirajRezervacijuKomanda objekat i dodeljuje mu jedinstveni IdZahteva. U ovom trenutku se sama rezervacija još ne upisuje u bazu. API upisuje samo ZahtevZaRezervaciju sa statusom NA_CEKANJU, a zatim RabbitMqPublisherService objavljuje komandu u durable red „kreiranje-rezervacije“. API korisniku vraća HTTP 202 Accepted i IdZahteva.

Frontend prelazi na stranicu statusa i periodično poziva /api/rezervacije/status/{idZahteva}. U međuvremenu Salon.Radnik, koji radi kao BackgroundService, preuzima poruku iz RabbitMQ reda i postavlja status zahteva na U_OBRADI.

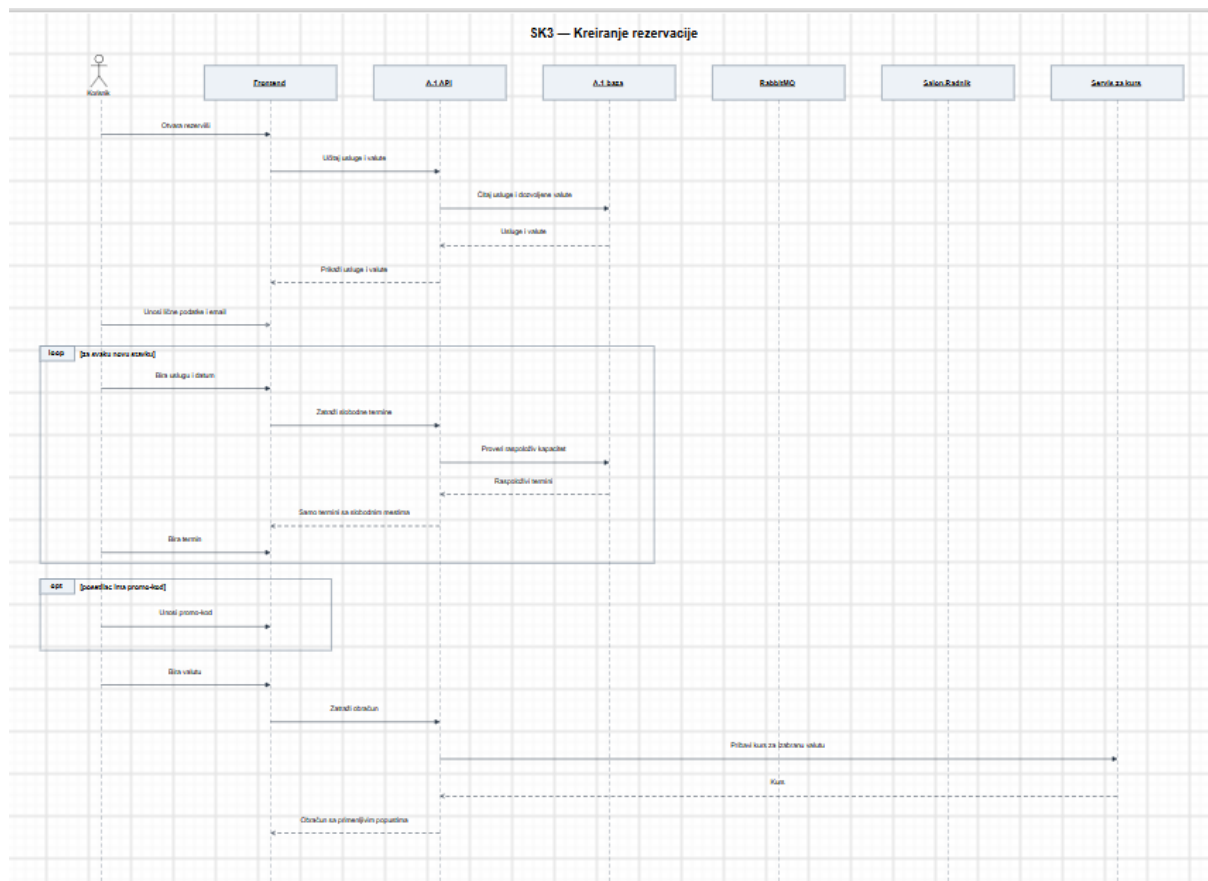
ObradaRezervacijeService vrši finalnu proveru. Proverava da rezervacija ima stavke, da izabrana valuta postoji u listi dozvoljenih valuta, da sve usluge postoje, da su izabrana vremena stvarni termini i da kapacitet nije prekoračen. Cene se ponovo čitaju iz baze, pa klijent ne može da nametne proizvoljnu cenu.

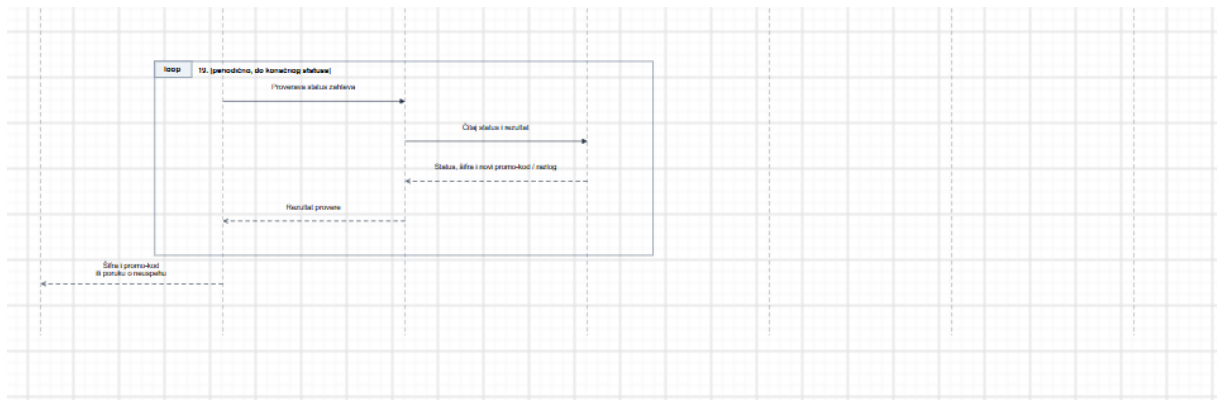
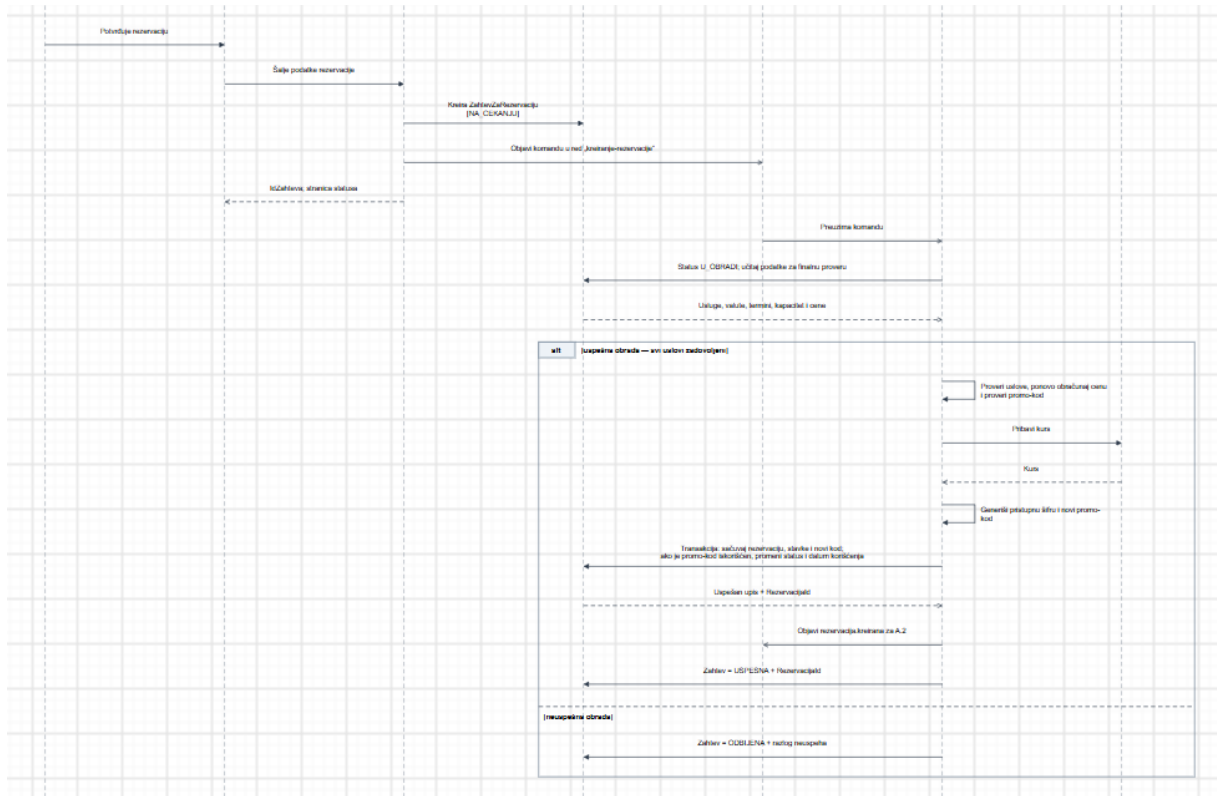
Zatim se izračunava ukupan iznos u RSD, primenjuje vremenski popust od 10% ako važi zadati datum i dodatnih 5% ako je promo-kod dostupan. KursService pribavlja kurs za konačan obračun u izabranoj valuti. GeneratorKodovaService generiše jedinstvenu pristupnu šifru i novi promo-kod.

Upis rezervacije, njenih stavki, promene statusa eventualno korišćenog promo-koda i kreiranje novog promo-koda izvršavaju se u okviru transakcije. Nakon uspešnog čuvanja Salon.Radnik poziva RezervacijaDogadjajService, koji formira RezervacijaKreiranaDogadjaj i preko topic exchange-a objavljuje događaj sa routing key-em rezervacija.kreirana.

Izvestavanje.Api je pretplaćen na rezervacija.# događaje. RabbitMqRezervacijaConsumer prima događaj, proverava DogadjajId radi idempotencije i poziva ObradaKreiraneRezervacijeService. Servis u A.2 bazu upisuje

Po uspešnoj obradi Salon.Radnik ažurira ZahtevZaRezervaciju na status USPESNA i upisuje Rezervacijald. Kada frontend sledeći put proverí status, Salon.Api vraća i pristupnu šifru i novi promo-kod, koji se prikazuju korisniku. Ako neka poslovna provera ne uspe, zahtev dobija status ODBIJENA i frontend prikazuje poruku o razlogu odbijanja.





5. Konačni dijagram klasa

Konačni dijagram klasa treba da bude izveden iz stvarne implementacije, a ne iz početnog plana. Zbog toga na dijagramu treba prikazati ključne klase po slojevima i njihove veze.

U domenskom delu A.1 ključne klase su OsnovneInformacije, KategorijaUsluge, Usluga, Rezervacija, StavkaRezervacije, PromoKod, DozvoljenaValuta i ZahtevZaRezervaciju. Potrebno je prikazati veze KategorijaUsluge–Usluga, Rezervacija–StavkaRezervacije, Usluga–StavkaRezervacije, Rezervacija–PromoKod i opcionu vezu ZahtevZaRezervaciju–Rezervacija.

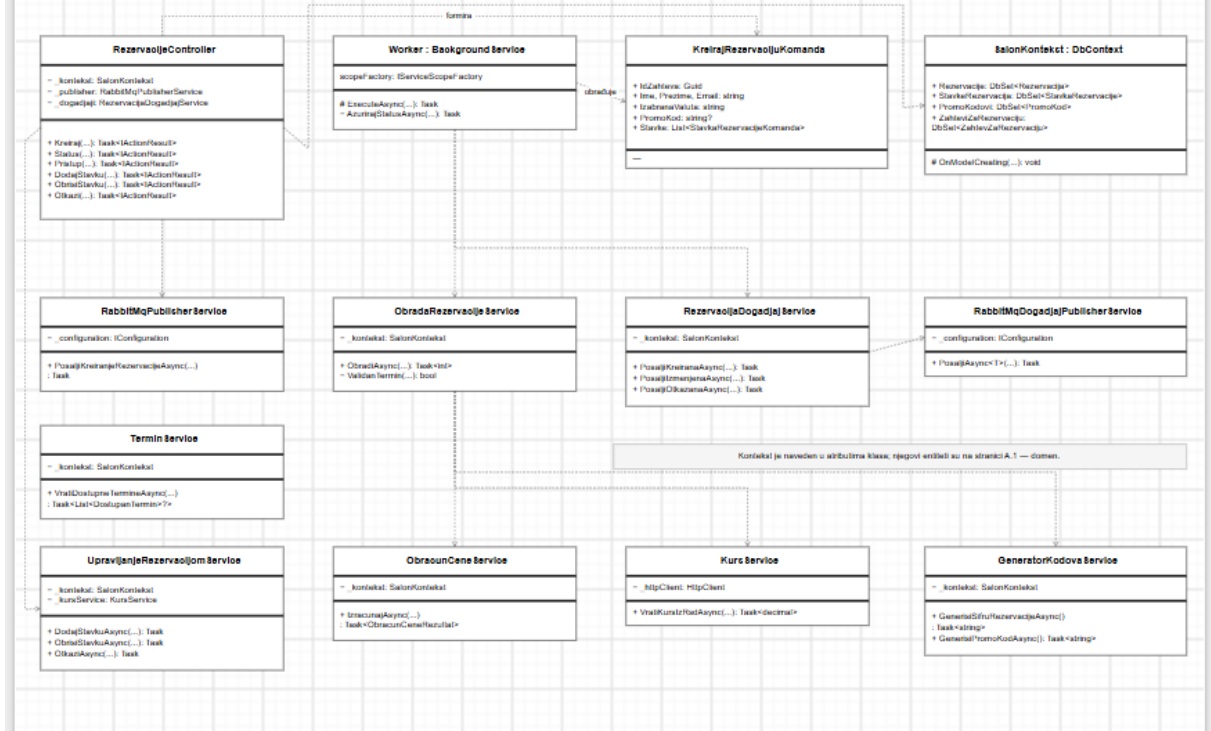
Na servisnom i aplikacionom nivou važni su TerminService, ObracunCeneService, KursService, GeneratorKodovaService, UpravljanjeRezervacijomService, ObradaRezervacijeService, RabbitMqPublisherService, RezervacijaDogadjajService i RabbitMqDogadjajPublisherService. RezervacijeController koristi servise za pristup i upravljanje rezervacijom, dok Salon.Radnik koristi ObradaRezervacijeService za asinhronu obradu komande iz RabbitMQ reda.

U zajedničkom projektu Zajednicko.Poruke treba prikazati poruke koje povezuju servise: KreirajRezervacijuKomanda i događaje RezervacijaKreiranaDogadjaj, RezervacijalzmenjenaDogadjaj i RezervacijaOtkazanaDogadjaj.

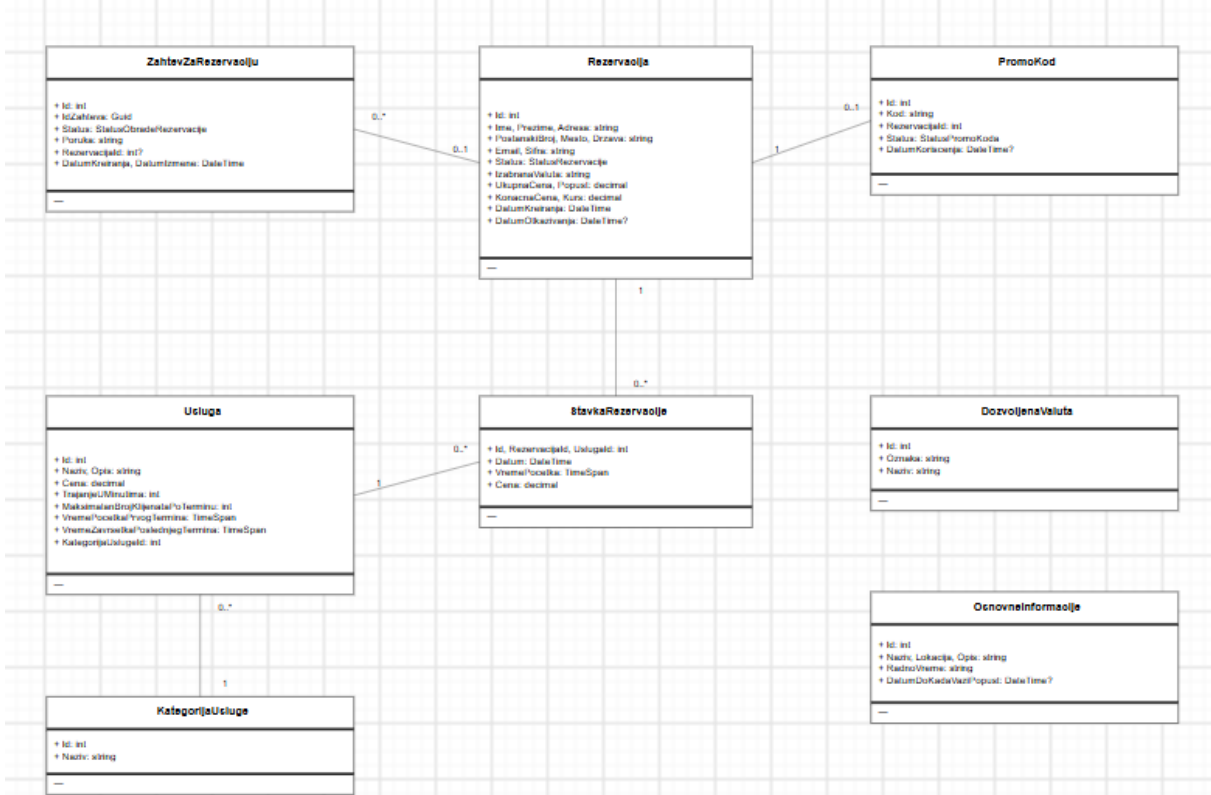
Za A.2 deo relevantne su klase IzvestajnaRezervacija, IzvestajnaStavkaRezervacije i ObradjenDogadjaj, kao i RabbitMqRezervacijaConsumer, IdempotencijaDogadjajaService, ObradaKreiraneRezervacijeService, ObradalzmenjeneRezervacijeService, ObradaOtkazaneRezervacijeService i IzvestajiController.

Dijagram treba da prikaže zavisnosti između kontrolera, servisa i konteksta baze, ali da ostane čitljiv. Frontend React komponente nije neophodno prikazivati na backend klasnom dijagramu, osim ako se želi jedan objedinjeni dijagram celog sistema.

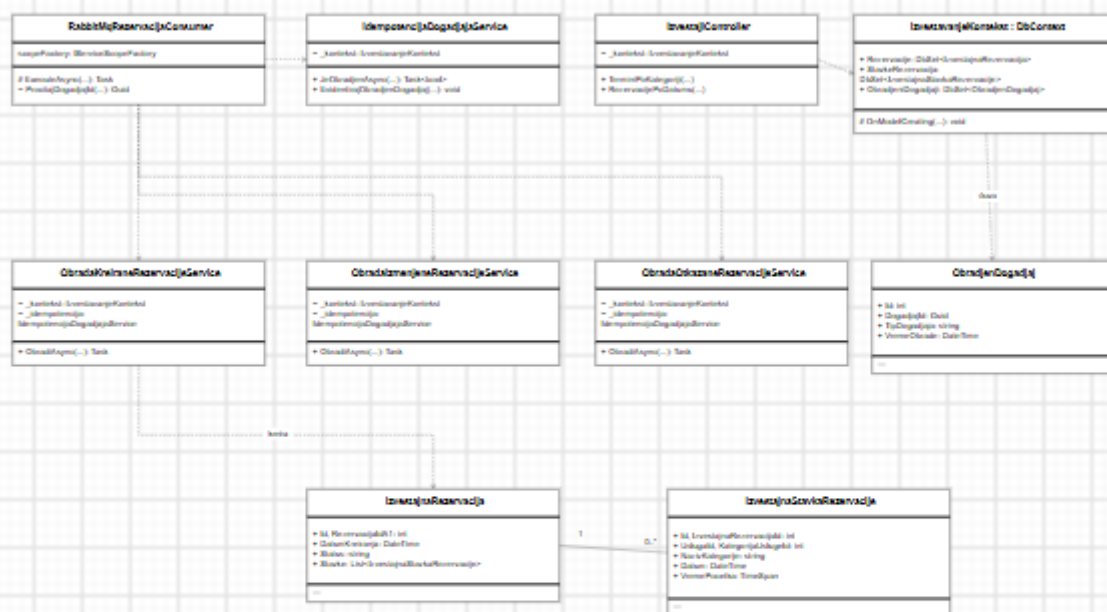
Salon Trač — A.1 — kontroleri i servisi



Salon Trač — A.1 — domenske klase



Salon Trač — A.2 — Izveštavanje i događaji



Zajedničke poruke — Zajedničke Poruke (klijenska svojstva)



6. Arhitektura sistema

Sistem je realizovan prema varijanti arhitekture C i sastoji se od više odvojenih aplikacionih celina povezanih REST pozivima i asinhronom event-driven komunikacijom preko RabbitMQ-a. A.1 predstavlja operativnu aplikaciju salona, dok A.2 predstavlja zaseban portal za izveštavanje sa sopstvenim backendom i sopstvenom bazom podataka.

6.1. Frontend sloj A.1 – salon-web

Korisnički interfejs A.1 realizovan je u React-u i TypeScript-u uz Vite. Aplikacija predstavlja single-page web aplikaciju i koristi React Router za navigaciju između početne strane, kreiranja rezervacije, praćenja statusa asinhronne obrade, pristupa postojećoj rezervaciji, upravljanja rezervacijom i administratorskih stranica.

Frontend sa backendom komunicira putem REST API-ja. Za kreiranje rezervacije ne čeka direktno završetak background obrade, već nakon HTTP 202 odgovora prelazi na stranicu statusa i periodično proverava rezultat obrade pomoću IdZahteva.

6.2. Backend sloj A.1 – Salon.Api

Salon.Api je ASP.NET Core Web API aplikacija na .NET 10 platformi. Kontroleri izlažu API za osnovne informacije, kategorije, usluge, valute, administrativna podešavanja, kurseve, dostupne termine, obračun cene i operacije nad rezervacijama.

Entity Framework Core i Pomelo MySQL provider koriste se za pristup relacionoj MySQL/MariaDB bazi. Poslovna logika je izdvojena u servisne klase u projektu Salon.Infrastruktura, dok se domenski entiteti i enumeracije nalaze u projektu Salon.Domen.

6.3. Background proces – Salon.Radnik

Salon.Radnik je .NET Worker Service koji je pretplaćen na RabbitMQ red za kreiranje rezervacija. Worker poruke obrađuje asinhrono i pre uzimanja naredne poruke završava obradu prethodne. On vrši finalne poslovne provere, poziva servis za obračun, pribavlja kurs, generiše kodove i tek nakon uspešnih provera upisuje rezervaciju u A.1 bazu.

Status asinhronog zahteva prolazi kroz stanja NA_CEKANJU, U_OBRADI i na kraju USPESNA ili ODBIJENA. Na taj način frontend može da prikaže stvarni rezultat background obrade bez blokiranja HTTP zahteva.

6.4. RabbitMQ komunikacija

RabbitMQ se koristi u dve svrhe. Prva je komanda za kreiranje rezervacije: Salon.Api objavljuje KreirajRezervacijuKomanda poruku u durable red „kreiranje-rezervacije“, a Salon.Radnik je njen potrošač. Druga je sinhronizacija A.1 i A.2 sistema kroz događaje. Nakon kreiranja, izmene ili otkazivanja rezervacije objavljuje se odgovarajući događaj sa routing key-em rezervacija.kreirana, rezervacija.izmenjena ili rezervacija.otkazana. Događaji se objavljuju na topic exchange, a A.2 red je vezan obrascem rezervacija.#.

6.5. Redis keš

Redis se koristi za keširanje osnovnih informacija salona. Pri čitanju se prvo proverava keš; ako podatak ne postoji, osnovne informacije se čitaju iz baze i čuvaju u Redis-u sa vremenskim ograničenjem. Nakon administratorske izmene osnovnih informacija odgovarajući keš ključ se briše, čime se sprečava prikaz zastarelih podataka.

6.6. Eksterni servis za kurs

Za preračunavanje cena koristi se otvoreni Frankfurter API. KursService dobija kurs iz RSD u izabranu dozvoljenu valutu. Kurs se pribavlja prilikom konačnog obračuna rezervacije, a vrednost kursa se čuva zajedno sa rezervacijom.

6.7. A.2 portal za izveštavanje

A.2 je fizički i logički odvojen od A.1. Frontend izveštavanje-web je zasebna React + TypeScript aplikacija, dok je Izveštavanje.Api zaseban ASP.NET Core backend sa svojom MySQL/MariaDB bazom. A.2 ne čita direktno A.1 bazu.

RabbitMqRezervacijaConsumer je background servis unutar A.2 backenda i pretplaćen je na događaje rezervacija.#. Za svaki događaj najpre proverava da li je DogadjajId već obrađen. Zatim poziva odgovarajući servis za kreiranje, izmenu ili otkazivanje izveštajne rezervacije i evidentira obrađeni događaj.

IzvestajiController iz A.2 baze formira dva tražena izveštaja: broj rezervisanih termina po kategoriji usluge i broj rezervacija po datumima kreiranja. A.2 frontend prikazuje oba izveštaja u tabelama i automatski ih osvežava na svakih pet sekundi.

6.8. Obrada događaja od A.1 do A.2

1. Kreiranje rezervacije završava se u Salon.Radnik procesu.
2. A.1 formira događaj koji sadrži trenutno stanje rezervacije i relevantne stavke.
3. Događaj se objavljuje na RabbitMQ topic exchange.
4. Izveštavanje.Api prima događaj preko svog reda.
5. A.2 proverava idempotenciju na osnovu DogadjajId.
6. Podaci se upisuju ili ažuriraju u A.2 bazi.
7. Portal pri sledećem automatskom osvežavanju prikazuje novo stanje izveštaja.

6.9. Tehnologije

- React + TypeScript + Vite – korisnički interfejsi A.1 i A.2.
- ASP.NET Core / .NET 10 – backend aplikacije.
- .NET Worker Service – background obrada novih rezervacija.
- Entity Framework Core + Pomelo – pristup MySQL/MariaDB bazama.
- MySQL/MariaDB – dve odvojene relacione baze, po jedna za A.1 i A.2.

- Redis – keš osnovnih informacija salona.
- RabbitMQ – asinhronne komande i događaji između podсистема.
- Frankfurter API – pribavljanje aktuelnog kursa valuta.